

- Variable names are case sensitive.
You can refer to the website for more information on the Ruby style guide.

Comments

- Single-line comments should be prefixed by an unquoted pound sign (#).
- Multi-line comments should be enclosed within `=begin` and `=end`.

Statement delimiters

Use a semicolon (;) to separate the multiple statements included in a single line.

Basic data types

Integers

- Numbers without a decimal point
- Small numbers or Fixnum. Example: 1, 3, -10004
- Big numbers or Bignum. Example: 49349443040329402394302483298439

Float

- Numbers with decimal points. Example: 0.4, 100.5

Strings

- This is an object that can comprise one or more bytes arranged in a random sequence, which usually represents characters, words or phrases. Example: "I am excited to learn Ruby!"

Boolean

- Boolean holds two values — true and false.

nil

- This is a keyword as well as an object, and it represents 'nothing'.
You will learn about a few other such data types as we explore other topics.

Operators and the input statement

Variable scope in Ruby: A scope helps you understand that part of the program in which there is a variable, beyond which it is not accessible.

- Global, class, instance and local are four different types of variable scopes in the Ruby program.
- You can identify a Ruby variable by the first character, which denotes the scope of the variable.
More about the types of scope will be discussed later in this article, when exploring 'class'.

Arithmetic operators and string arithmetic

Arithmetic operators: There are six arithmetic operators:

- Addition (+)

- Subtraction (-)
- Multiplication (*)
- Division (/)
- Exponentiation (**)
- Modulo (%).

Here is an example.

- Exponentiation: `3**3` is 27
- Modulo `25 % 8` would be 1

String arithmetic: Strings can be used with arithmetic operators. For example:

- `puts 'I like' + 'Ruby programming.'` will concatenate the strings displaying the output as 'I like Ruby programming'.
- `puts like 'Ruby ' * 5` will print Ruby five times.

Comparison operators: Ruby provides many comparison operators to check the equality between two values. These are shown below:

- `==` Tests for equality
- `.eql?` Same as `==`
- `!=` Tests for inequality
- `<` Less than
- `>` Greater than
- `>=` Greater than/equal to
- `<=` Less than/equal to
- `<=>` Combined comparison operator

Conversions:

- The string version of an object can be acquired by adding `'.to_s'` after the object.
- The integer version of an object can be acquired by adding `'.to_i'` after the object.
- The float version of an object can be derived by adding `'.to_f'` after the object.

An example is shown below:

```
num1 = 3
num2 = '15'
puts num1.to_s + num2
puts num1 + num2.to_i
Output:
315
18
```

Getting user inputs: The user input in a Ruby program can be acquired by using `'gets'`, as shown below:

```
puts 'Enter your name'
name = gets
name = name.chomp
puts 'Hello ' + name
```

Here, `'chomp'` is a string method, which will omit all the trailing characters after the space.